

University of Kassel
Department of Electrical Engineering and Computer Science
Degree Programme Computer Science, B.Sc.

Learning Heuristics for Efficient Causal Question Answering Using A* Search

Bachelor's Thesis

Ashkan Kiafard
Born Jun 23, 2002 in Iran

Matriculation Number 35825953

1. Referee: Prof. Dr. Martin Potthast
2. Referee: Prof. Dr. Oliver Hohlfeld

Submission date: September 9, 2026

Declaration

Unless otherwise indicated in the text or references, this thesis is entirely the product of my own scholarly work.

Kassel, September 9, 2026

.....
Ashkan Kiafard

Abstract

Many information needs are causal, asking why events occur, what consequences they have, or whether one event leads to another. Such questions arise across domains including healthcare, economics, public policy, and environmental science. A fundamental task is answering the binary question “Does A cause B ?” While large language models can answer such questions directly, their responses may hallucinate causal relationships and often lack an explicit causal path. Causal graphs instead encode causal knowledge explicitly by representing concepts or events as nodes and causal relationships as directed edges. This allows binary causal question answering to be formulated as directed pathfinding, where an answer can be supported by an explicit path from A to B . Exhaustive pathfinding methods such as Breadth-First Search (BFS) become expensive on large causal graphs. This thesis proposes an embedding-guided A^* method for efficient causal path discovery. Embedding distances are used as edge costs and heuristic estimates toward the target concept. Off-the-shelf sentence embeddings already provide useful search guidance, while fine-tuning further reduces search effort. Matryoshka Representation Learning enables compact embeddings and allows the trade-off between search efficiency and effectiveness to be controlled through embedding dimensionality. Experiments on binary causal question answering datasets derived from MS MARCO and SemEval across CauseNet Precision, CauseNet Full, and CEG Filtered show efficiency gains. On the larger CauseNet Full and CEG Filtered graphs, the selected A^* configuration is at least $1.3\times$ as fast as capped BFS and $8.2\times$ as fast as uncapped BFS, while reducing the number of visited nodes by factors of at least 5.9 and 107, respectively. Compared with the reinforcement learning approach of Blübaum and Heindorf, it achieves higher F_1 in all final test settings while being at least $56\times$ as fast. Overall, the results show that learning an embedding-based heuristic provides an efficient alternative to exhaustive and neural graph traversal.

Contents

1	Introduction	1
2	Foundations	3
2.1	Causal Graphs and Binary Causal Questions	3
2.2	Heuristic Search and the A^* Algorithm	4
2.3	Representation Learning for Graph Search	5
2.4	Matryoshka Representation Learning	6
2.5	Activation Functions	6
3	Related Work	8
3.1	Causal Question Answering	8
3.2	Causal Knowledge Graphs	9
3.2.1	CauseNet	9
3.2.2	CausalBank and CEG	10
3.3	RL Agent for Graph Traversal	11
3.4	Learned Heuristics for Search	11
4	Methodology	12
4.1	Problem Setting	12
4.2	Embedding-Guided A^* Search	12
4.3	Fine-Tuning the Embedding Space	13
4.4	Implementation Overview	14
5	Training Procedure	15
5.1	Supervision Construction	15
5.2	Ranking-Based Training Objective	16
5.3	Matryoshka Representation Learning	17
5.4	Validation Procedure	18
5.5	Hyperparameter Optimization	18
5.6	Final Model Training	19
5.7	Final Embedding Precomputation	19

6	Results	20
6.1	Evaluation Setup	20
6.1.1	Evaluation Procedure	20
6.1.2	Datasets	21
6.1.3	Causal Graphs	21
6.1.4	Models and Baselines	22
6.1.5	Evaluation Methodology	23
6.2	Effect of Fine-Tuning	25
6.3	Effect of Matryoshka Dimensionality	27
6.4	Final Test Results	29
6.5	Activation and Distance Ablation	30
7	Discussion	33
8	Conclusion	34
	Bibliography	35
A	Algorithmic Details	45

Chapter 1

Introduction

Many questions about real-world phenomena are causal in nature, from medical diagnosis [2] to analyzing the effects of policies [3] or environmental changes [4]. Humans can approach such questions using domain knowledge, experience, and intuition. Answering them automatically, however, requires causal knowledge in a machine-processable form. While causal information can be extracted from large text collections, its unstructured nature makes algorithmic reasoning difficult. Causal graphs provide a structured representation using nodes for concepts or events and directed edges for causal relationships, enabling causal reasoning via graph traversal. The binary question of whether a concept A causes another concept B can therefore be formulated as a pathfinding problem from A to B . Causal graphs such as CauseNet [5] contain millions of cause-effect relationships extracted from the web, making efficient traversal a challenge.

A straightforward approach is exhaustive search using Breadth-First Search (BFS), which guarantees completeness and shortest paths in terms of hop count [6]. However, shortest paths are not necessarily the most semantically meaningful. Due to the noisy, automatically extracted nature of large causal graphs, they may include weak or overly general causal links, limiting their usefulness as explanations. For example, in CauseNet, BFS connects *waves* to *destruction* through the path *waves* $\rightarrow$ *death* $\rightarrow$ *destruction*. Although topologically valid, the highly connected intermediate concept *death* provides only a broad connection between the queried concepts. In addition, BFS becomes computationally expensive in graphs with high branching factors, often requiring the exploration of thousands of nodes before reaching the target. To reduce this search space, a reinforcement learning (RL) traversal approach has been proposed [1]. This approach learns a policy to guide exploration toward promising regions of the graph, substantially reducing the number of visited nodes. However, maintaining and evaluating a dedicated neural policy introduces architectural complexity and inference overhead.

To make causal question answering more efficient, this thesis replaces the learned traversal policy with a learned search heuristic, removing neural computations from graph traversal at inference time. The proposed approach employs A^* search with the embedding distance between the current and target nodes as its heuristic. A pretrained sentence transformer provides embeddings that guide the search toward semantically closer nodes, reducing unnecessary exploration. Since semantic proximity alone does not necessarily identify the most promising successor, the embeddings are further fine-tuned for the search objective. Training examples are derived from A^* paths that reach the target, teaching the model to favor successors that provide a shorter embedding-space route toward the target. In the previous example, A^* with the fine-tuned embeddings instead finds the equally short path *waves* $\rightarrow$ *turbulence* $\rightarrow$ *destruction*, replacing the broad intermediate concept *death* with the more specific *turbulence*. Empirical evaluation shows that fine-tuning substantially reduces the number of visited nodes while generally retaining competitive effectiveness. BFS remains more effective in some settings, but its search effort increases substantially on larger graphs. Compared with the RL method of Blübaum and Heindorf [1], the proposed approach achieves F_1 scores at least 7.3 percentage points higher across all final test settings while being at least $56\times$ as fast.

The contributions of this thesis are as follows: (1) an embedding-guided A^* approach is proposed for efficient binary causal question answering over large-scale causal graphs; (2) a search-oriented fine-tuning procedure adapts the embedding space for causal pathfinding, together with Matryoshka Representation Learning to support efficient low-dimensional heuristics; and (3) a systematic evaluation on in-domain and out-of-domain causal graphs and datasets compares the approach with BFS and RL traversal, demonstrating substantial reductions in search effort and runtime while characterizing the resulting effectiveness–efficiency trade-off.

The remainder of this thesis is structured as follows: The theoretical foundations required for the proposed approach are introduced in Chapter 2. Related work on causal question answering, causal graphs, and graph traversal methods, including RL-based approaches and learned search heuristics, is reviewed in Chapter 3. Chapter 4 then presents the problem formulation and the proposed embedding-guided A^* approach for causal graph traversal. The training procedure, including supervision construction, fine-tuning, validation, and model selection, is described in Chapter 5. Experimental evaluation and comparisons with BFS and RL across different datasets and causal graphs follow in Chapter 6. Chapter 7 discusses the main findings, trade-offs, and limitations. Finally, Chapter 8 summarizes the thesis and outlines future work.

Chapter 2

Foundations

This chapter introduces the theoretical concepts required for the remainder of this thesis. Section 2.1 formalizes causal graphs and frames binary causal questions as reachability problems. Section 2.2 presents heuristic search and the A^* algorithm. Section 2.3 introduces transformer-based embedding models and their use for graph search. Section 2.4 presents Matryoshka Representation Learning. Finally, Section 2.5 introduces the non-linear activation functions used within the training objective.

2.1 Causal Graphs and Binary Causal Questions

Causal graphs provide a structured representation of causal relationships that would otherwise remain implicit in unstructured text [5, 7]. They can be modeled as directed graphs $G = (V, E)$, where nodes represent concepts or events and directed edges encode causal relations between them. This structure enables algorithmic reasoning over causal dependencies. Figure 2.1 illustrates three common causal structures, namely a causal chain, a common cause, and a common effect. Given two concepts A and B , let $a, b \in V$ denote their corresponding graph nodes. The binary question “Does A cause B ?” then reduces to determining whether a directed path exists from a to b [1]:

$$\exists (a = v_0, v_1, \dots, v_k = b) \text{ such that } \forall i \in \{0, \dots, k-1\} (v_i, v_{i+1}) \in E.$$

If such a path exists, the query is answered positively under this graph-based formulation. If no path can be found, either no causal relation is encoded in the graph or the available knowledge is incomplete. This formulation enables the application of classical graph traversal algorithms for causal reasoning.

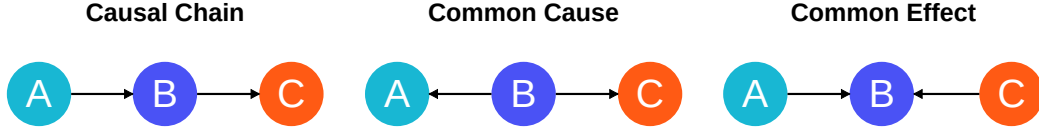


Figure 2.1: Three common causal structures in directed graphs, showing a causal chain, a common cause, and a common effect.

2.2 Heuristic Search and the A^* Algorithm

Graph traversal can be performed using uninformed or informed search algorithms. Uninformed search methods, such as BFS, explore the graph without guidance and guarantee completeness [6]. However, their computational cost can be high in large graphs with significant branching factors. Heuristic search algorithms incorporate problem-specific information to guide exploration. The A^* algorithm is an informed best-first search strategy [8] that selects nodes based on an evaluation function

$$f(n) = g(n) + h(n),$$

where $g(n)$ denotes the cost of the path from the start node to node n , $h(n)$ is a heuristic estimate of the remaining cost from n to the target node, and $f(n)$ represents the estimated total cost of a solution path passing through n . At each step, A^* expands the node with the smallest $f(n)$ value. The properties of A^* depend on the quality of the heuristic function. Two important properties are consistency and admissibility [8]. A heuristic is *consistent* if, for every node n and each successor s , it satisfies

$$h(n) \leq c(n, s) + h(s),$$

where $c(n, s)$ is the cost of transitioning from n to s . Consistency implies that f -values are non-decreasing along every path. Consequently, once a node has been expanded, it does not need to be reopened. A weaker condition is *admissibility*, which requires that the heuristic never overestimates the true remaining cost:

$$h(n) \leq d(n),$$

where $d(n)$ denotes the optimal cost from node n to the goal. If the heuristic is admissible, A^* is guaranteed to find an optimal path when closed nodes can be reopened after a cheaper path is discovered. Without consistency, such re-expansions may be necessary.

2.3 Representation Learning for Graph Search

Modern natural language processing systems commonly represent textual concepts as dense vector embeddings. Transformer-based architectures form the basis of many state-of-the-art embedding models. A transformer encoder maps an input text sequence to contextualized token representations through self-attention, which allows each token to incorporate information from other tokens in the sequence based on their relevance [9]. Sentence transformer models aggregate contextualized token representations into fixed-dimensional embeddings, forming a geometric space in which semantically related concepts tend to be located closer together [10]. In graph search settings, such geometric structure can be used to guide exploration toward promising regions of the graph. Given two embeddings $\mathbf{x}, \mathbf{y} \in \mathbb{R}^D$, the distance between them can be measured using cosine distance or Euclidean distance. Cosine distance is derived from cosine similarity [11], which is defined as

$$\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}, \quad \|\mathbf{x}\|, \|\mathbf{y}\| \neq 0.$$

It lies in the interval $[-1, 1]$. The corresponding cosine distance is

$$d_{\text{Cos}}(\mathbf{x}, \mathbf{y}) = 1 - \text{cosine}(\mathbf{x}, \mathbf{y}),$$

which lies in $[0, 2]$. The second distance measure considered in this work is Euclidean distance [11], defined as

$$d_{\text{Euc}}(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2.$$

In this thesis, an embedding-guided variant of A^* search is employed. Let $\phi(v)$ denote the embedding of node v . The cost of traversing an edge from node n to a successor s is defined as the distance between their embeddings, while $g(n)$ is the sum of these edge costs along the path from the start node to n . The heuristic $h(n)$ is the embedding distance from n to the target node b . When Euclidean distance is used, consistency follows directly from the triangle inequality. For every successor s of node n ,

$$h(n) = \|\phi(n) - \phi(b)\|_2 \leq \|\phi(n) - \phi(s)\|_2 + \|\phi(s) - \phi(b)\|_2 = c(n, s) + h(s).$$

The heuristic therefore satisfies the consistency condition of A^* . In contrast, cosine distance does not generally satisfy the standard triangle inequality and therefore does not guarantee heuristic consistency [12]. Since this work uses embedding distance for both the edge cost and the heuristic, consistency guarantees optimality only with respect to the accumulated distance in embedding space. However, this quantity has no clear interpretation in terms of causal-path quality or hop count. As the practical objective is to efficiently discover valid causal paths, with short and meaningful paths being desirable but not directly optimized, consistency is of limited practical relevance.

2.4 Matryoshka Representation Learning

Standard embedding models are typically trained to produce representations with a fixed dimensionality. If such an embedding is truncated to a smaller number of dimensions, there is no guarantee that the reduced vector retains meaningful structure. Matryoshka Representation Learning [13] addresses this limitation by optimizing multiple prefix subspaces of an embedding to remain informative. Let $\mathbf{e} \in \mathbb{R}^D$ denote a learned embedding vector with full dimensionality D . Instead of optimizing only the full representation, the model is trained such that several nested prefix dimensions

$$m_1 < m_2 < \dots < m_k = D$$

preserve useful structural information. For each dimension m_i , the corresponding truncated embedding is defined as

$$\mathbf{e}_{m_i} = \mathbf{e}_{1:m_i},$$

where $\mathbf{e}_{1:m_i}$ denotes the prefix consisting of the first m_i components of the full embedding vector. The training objective is applied at multiple prefix dimensions, encouraging each truncated embedding to retain task-relevant information. This promotes an ordered representation in which earlier dimensions capture broadly useful features, while later dimensions add finer-grained information and additional capacity [13]. This property enables a trade-off between representational capacity and computational efficiency. In applications such as graph search, reducing embedding dimensionality can lower the cost of distance computations while preserving sufficient information for effective heuristic guidance. However, overly small prefixes may discard too much information and degrade the quality of the heuristic.

2.5 Activation Functions

Neural networks rely on non-linear activation functions to model complex relationships between inputs and outputs. Without non-linearities, stacked linear transformations would collapse into a single linear mapping, severely limiting representational power. One widely used activation function is the Rectified Linear Unit (ReLU) [14], defined as

$$\text{ReLU}(x) = \max(0, x).$$

ReLU maps negative inputs to zero while preserving positive inputs. Its simple piecewise-linear form allows for efficient computation. Another commonly used

activation function is the Gaussian Error Linear Unit (GELU) [15], defined as

$$\text{GELU}(x) = \frac{x}{2} \left(1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right).$$

Here, erf denotes the error function. A commonly used tanh-based approximation is [15]

$$\text{GELU}(x) \approx \frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right).$$

This formulation provides a computationally efficient approximation while preserving the smooth gating behavior of GELU. Unlike ReLU, which applies a hard threshold and is not differentiable at $x = 0$, GELU provides a smooth activation that weights inputs according to their magnitude. Figure 2.2 illustrates this difference. In this thesis, ReLU and GELU are evaluated as non-linear transformations within the training objective. Their different shapes determine how score differences are transformed and how strongly they contribute to the loss, thereby shaping the learned embedding space used by the A^* heuristic.

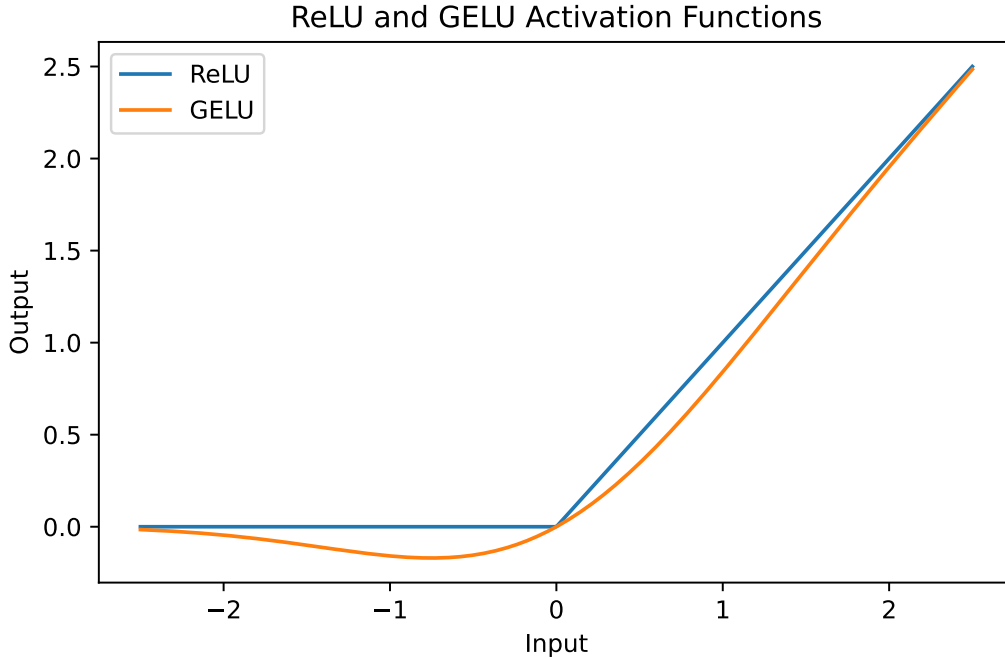


Figure 2.2: Comparison of ReLU and GELU activation functions.

Chapter 3

Related Work

This chapter reviews the main prior work on which this thesis builds. Section 3.1 introduces causal question answering and distinguishes between text-based and graph-based approaches. Section 3.2 discusses causal knowledge graphs, their construction, and their properties. Subsections 3.2.1 and 3.2.2 present CauseNet and the Cause Effect Graph (CEG), the causal graph resources used in this work. Section 3.3 discusses the current state of the art in causal graph traversal, a reinforcement learning (RL) approach that addresses the same binary causal question answering task and therefore provides the most relevant point of comparison for this thesis. Finally, Section 3.4 reviews prior work on learned heuristics for A^* search and discusses how it relates to the causal graph traversal setting considered in this thesis.

3.1 Causal Question Answering

Causal questions concern why events occur, what consequences they produce, whether one event causes another, and how outcomes would differ under interventions or counterfactual conditions [16, 17]. They therefore cover a range of tasks, from identifying causes and effects to reasoning about causal direction and hypothetical outcomes. The task is relevant in practice, as Bondarenko et al. [16] estimate that at least 5% of search-engine queries are causal. This thesis focuses on binary causal questions of the form “Does A cause B ?”

Methods for causal question answering can broadly be divided into text-based and graph-based approaches. Text-based methods infer causal relationships from patterns, retrieved evidence, statistical associations, or neural language models. Although large language models perform well on some causal reasoning benchmarks [18], controlled evaluations report only a preliminary understanding of causal graphs [19], frequent confusion between causal and temporal relations [20], and substantial drops in effectiveness on fresh,

nearly unseen data [21]. These findings suggest that current models often recall learned causal associations more reliably than they perform systematic causal reasoning [22]. Graph-based approaches instead operate on explicitly represented causal relationships, enabling algorithmic reasoning through methods such as graph traversal.

3.2 Causal Knowledge Graphs

Knowledge graphs provide structured representations of entities or concepts and their relationships. General knowledge graphs such as Wikidata [23], DBpedia [24], and ConceptNet [25] cover diverse relation types, whereas causal knowledge graphs focus specifically on cause–effect relationships.

Causal knowledge graphs may be created manually or automatically. Manual construction provides greater control over relationship quality but scales poorly, whereas automatic extraction enables large-scale coverage at the cost of increased noise. Prominent automatically extracted general-domain examples are CauseNet [5] and CEG [7]. In contrast, domain-specific resources provide more constrained vocabularies and richer domain context at the cost of narrower coverage [26, 27]. Causal knowledge graphs also differ in their level of normalization. Strongly normalized resources represent entities in structured and canonical forms, whereas weakly normalized resources represent textual concepts, phrases, or lexical terms more directly. CauseNet uses textual expressions as graph nodes, whereas CEG represents causal relationships between lemmatized terms rather than canonical entities [5, 7]. Strong normalization improves consistency and reduces duplication, while weak normalization preserves a broader range of linguistic expressions at the cost of greater ambiguity.

These characteristics influence both the quality and scale of causal knowledge graphs. In large, automatically constructed graphs, their size, noise, and weakly normalized representations can make efficient and reliable graph traversal particularly challenging.

3.2.1 CauseNet

CauseNet [5] is a large-scale causal knowledge graph containing millions of cause–effect relationships extracted from web data. It was constructed by identifying linguistic patterns that express causality and applying them to semi-structured and unstructured text sources. The resulting graph contains over 11 million causal relationships with an estimated precision of approximately 83%. Each directed edge connects two textual concepts and contains provenance information that allows the represented causal relationship to be traced back to

its source sentence. Since the graph is constructed automatically, it contains noisy and potentially spurious relationships. CauseNet therefore also provides a high-precision subset containing approximately 200,000 relationships with a reported precision of 96% [5]. The full graph offers broader coverage, whereas the high-precision subset prioritizes relationship quality. Figure 3.1 shows an example excerpt from CauseNet.

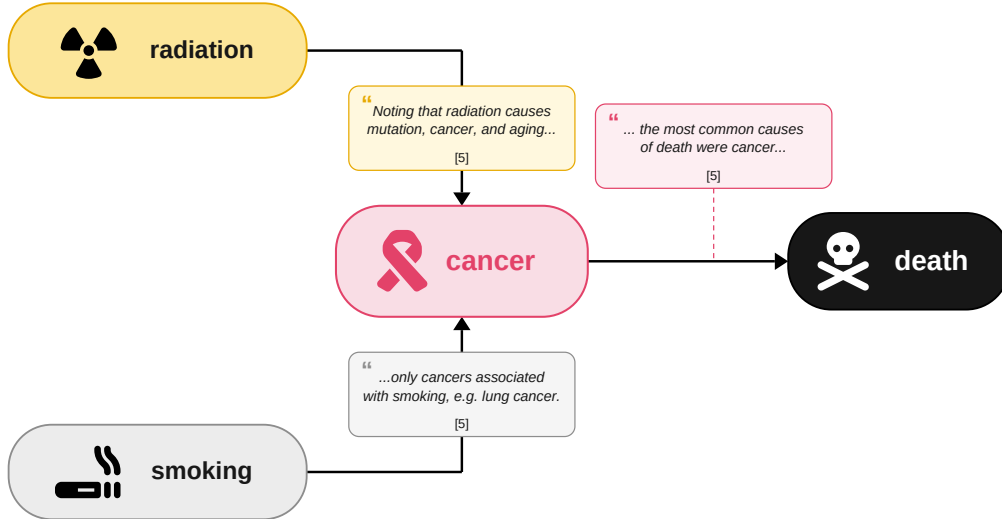


Figure 3.1: Example excerpt from CauseNet showing causal relations between concepts such as radiation, smoking, cancer, and death [5].

3.2.2 CausalBank and CEG

Li et al. [7] introduce two related open-domain causal resources extracted automatically from the English Common Crawl corpus, namely CausalBank and CEG. CausalBank is a sentence-level corpus containing approximately 314 million cause–effect pairs identified using lexical patterns that explicitly express causal relationships. CEG aggregates these extracted relationships into a directed graph between lemmatized terms. As this thesis investigates graph traversal, it uses CEG rather than the sentence-level CausalBank corpus. Each edge records the occurrence frequency of the corresponding term pair together with necessity and sufficiency causality scores. Li et al. report approximately 89.1 million relationships after filtering those occurring five times or less [7].

3.3 RL Agent for Graph Traversal

Blübaum and Heindorf [1] propose a reinforcement learning (RL) approach to binary causal question answering through graph traversal. Their agent is trained on CauseNet to learn a policy that maps the query and current graph node to a distribution over neighboring nodes. During inference, multiple paths are sampled, and a causal relationship is predicted if and only if one reaches the target node. By directing traversal toward promising neighbors, the approach reduces the number of explored nodes by up to 99% compared with BFS while maintaining comparable accuracy [1]. Despite these gains, the method requires training an RL agent and repeatedly evaluating its policy during traversal, resulting in additional computational overhead.

3.4 Learned Heuristics for Search

Informed search algorithms such as A^* rely on heuristic functions to estimate the remaining cost from a current state to the goal. Designing effective heuristics manually often requires substantial domain knowledge and problem-specific experimentation, which has motivated approaches that learn these estimates from data. Numeroso et al. [28] propose a neural method for learning consistent heuristic functions that can be integrated directly into A^* search. On synthetic pathfinding tasks, their learned heuristics reduce search effort compared with Dijkstra’s algorithm while preserving optimality. This thesis instead investigates learned heuristics for binary causal question answering over large-scale causal knowledge graphs.

Chapter 4

Methodology

This chapter presents the proposed methodology for binary causal question answering over directed causal knowledge graphs. Section 4.1 specifies the graph-based problem setting considered in this work. Section 4.2 introduces the bounded embedding-guided A^* search procedure, including its traversal cost and heuristic functions. Section 4.3 motivates the fine-tuning of the embedding space for more effective search guidance. Finally, Section 4.4 provides an overview of the main software components and interactive demonstration.

4.1 Problem Setting

Following the graph-based formulation introduced in Section 2.1, this thesis treats binary causal question answering as directed pathfinding. Given two concepts A and B with corresponding graph nodes $a, b \in V$, the task is to find a directed path from a to b . If found, the system answers that a causal relationship exists. Otherwise, it answers that none exists.

4.2 Embedding-Guided A^* Search

To search efficiently for a directed causal path from a to b , this work employs an embedding-guided A^* algorithm. In spatial pathfinding applications, A^* heuristics often rely on geometric distances, whereas causal knowledge graphs provide no spatial coordinates from which such estimates can be derived. Therefore, each graph node is mapped to a corresponding point in a D -dimensional embedding space according to

$$\phi: V \rightarrow \mathbb{R}^D,$$

where $\phi(v)$ denotes the vector representation of node v . The traversal cost of an edge (u, v) is defined as the distance between the embeddings of u and v ,

$$c(u, v) = d(\phi(u), \phi(v)),$$

where $d(\cdot, \cdot)$ denotes either cosine or Euclidean distance. For a node n , the accumulated cost from the start node a is defined as

$$g(n) = \sum_{(u,v) \in P_{a,n}} c(u, v),$$

where $P_{a,n}$ denotes the currently known path from a to n . The heuristic function estimates the remaining distance from n to the target node b as

$$h(n) = d(\phi(n), \phi(b)),$$

and the priority of each node is determined by

$$f(n) = g(n) + h(n).$$

Let τ denote the maximum visited-node budget, with $\tau = -1$ indicating that no limit is applied. Nodes are expanded in ascending order of $f(n)$ until b is reached, the priority queue is exhausted, or $\tau + 1$ nodes have been visited. The implementation retains the lowest known path cost for each node and does not reopen nodes after expansion. This reduces traversal overhead but forfeits the optimality guarantee of classical A^* when the heuristic is inconsistent. Algorithms 1 and 2 present the reachability-only and path-returning variants, respectively. The former is used for efficient evaluation, while the latter is used when the discovered path is needed.

4.3 Fine-Tuning the Embedding Space

The evaluated pretrained embedding models are designed primarily for semantic similarity, retrieval, or ranking rather than causal graph traversal [29–33]. Their original embedding spaces therefore do not necessarily reflect which neighboring nodes are most relevant for reaching a causal target. To improve search guidance, the embedding space is fine-tuned using successful causal search paths. Because the same embedding distance defines both the edge cost and the heuristic, fine-tuning adapts both components of the A^* evaluation function. At each step, the successor on the discovered path is contrasted with alternative outgoing neighbors, encouraging the model to assign more favorable embedding-space distances to transitions that lead toward the target. The resulting representations are therefore better aligned with the requirements of embedding-guided A^* search. Chapter 5 details this procedure.

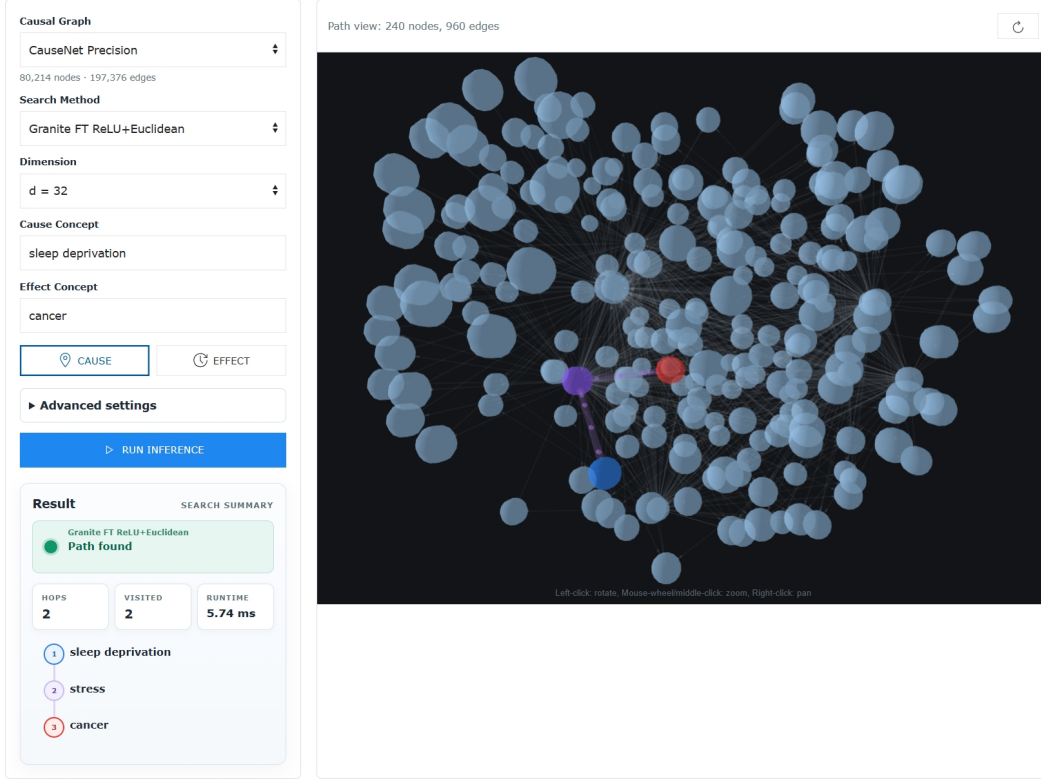


Figure 4.1: Web interface for configuring embedding-guided A^* search and visualizing the resulting causal path.

4.4 Implementation Overview

The proposed methodology is implemented as a pipeline that integrates graph search, hyperparameter optimization, model training, evaluation, and interactive visualization. Dataset processing relies on Hugging Face Datasets [34], while Optuna [35] is used for hyperparameter optimization before training. The sentence transformer models [10] are trained with PyTorch [36] and PyTorch Lightning [37], while NumPy [38] is used for embedding storage and access. The demonstration shown in Figure 4.1 is implemented as an interactive web application using FastAPI [39] and 3d-force-graph [40]. It allows users to select a graph, embedding model, and Matryoshka dimension, execute embedding-guided A^* search, and visualize the resulting causal path.¹

¹The demonstration is deployed at <https://causalpathfinding.web.webis.de/>.

Chapter 5

Training Procedure

This chapter describes the procedure used to adapt pretrained embedding models for more efficient A^* traversal. Section 5.1 explains how successful graph traversals are converted into supervised preference examples. Sections 5.2 and 5.3 introduce the ranking-based training objective and its extension to Matryoshka Representation Learning. Section 5.4 presents the search-based validation procedure, while Sections 5.5 and 5.6 describe hyperparameter optimization and final model training. Finally, Section 5.7 explains how the trained embeddings are precomputed and stored for evaluation and inference.

5.1 Supervision Construction

To fine-tune the embedding models, supervised training and validation datasets are constructed from the corresponding splits of the binary causal question dataset derived from MS MARCO by Blübaum and Heindorf [1]. This construction is specific to model training and hyperparameter optimization, while the subsequent evaluation data remain unchanged. Each pretrained base model is used to generate its own supervision.

To construct a dataset for a given split, consider a causal graph $G = (V, E)$. Graph-node embeddings are first precomputed using the respective pretrained base model. For each cause–effect pair (A, B) in the split, let $a, b \in V$ denote the corresponding graph nodes. An unbounded A^* search is performed from a to b using distance metric $d \in \{\text{Cos}, \text{Euc}\}$, denoting cosine and Euclidean distance, respectively. If the search reaches the target b , the discovered path is retained. Let

$$P = (p_0, \dots, p_k), \quad p_0 = a, \quad p_k = b, \quad k \geq 1,$$

denote such a path, where p_i is the node at position i and k is the number of edges in the path. The A^* path is used instead of a minimum-hop path ob-

tained with BFS because minimizing the number of edges does not necessarily yield a path useful for embedding-guided search. In contrast, A^* prioritizes nodes according to the embedding-space edge costs and heuristic estimates of the pretrained model. The resulting paths therefore provide supervision aligned with the search procedure that fine-tuning aims to improve.

For each distance metric d , let $\mathcal{D}_{\text{train}}^d$ and $\mathcal{D}_{\text{val}}^d$ denote the datasets constructed from the training and validation splits, respectively. Let $N^+(p_i)$ denote the set of outgoing successors of node p_i . For every step along a retained path, the path successor p_{i+1} is treated as the preferred choice and compared with each alternative successor $n \in N^+(p_i) \setminus \{p_{i+1}\}$. For either split $s \in \{\text{train}, \text{val}\}$, the corresponding quadruples are added as

$$\mathcal{D}_s^d \leftarrow \mathcal{D}_s^d \cup \{(p_i, p_k, p_{i+1}, n) \mid 0 \leq i < k, n \in N^+(p_i) \setminus \{p_{i+1}\}\}.$$

Each quadruple is subsequently denoted by (c, e, p, n) , representing the current node, target node, preferred successor on the discovered path, and alternative successor, respectively. It defines a pairwise preference that encourages the model to assign a lower estimated traversal cost to the preferred successor than to the competing choice. Algorithm 3 summarizes the construction procedure. The resulting datasets are cached after construction and reused throughout hyperparameter optimization and final model training, avoiding repeated A^* searches for generating the same supervision.

5.2 Ranking-Based Training Objective

The ranking-based objective uses quadruples $(c, e, p, n) \in \mathcal{D}_{\text{train}}^d$ to reshape the embedding space for graph traversal. The preferred successor p should yield a lower cumulative distance to the target e than the alternative successor n . The difference between the preferred and alternative traversal costs is defined as

$$\begin{aligned} \Delta &= f(p) - f(n) \\ &= [g(p) + h(p)] - [g(n) + h(n)] \\ &= [d(\phi(c), \phi(p)) + d(\phi(p), \phi(e))] - [d(\phi(c), \phi(n)) + d(\phi(n), \phi(e))]. \end{aligned}$$

A value $\Delta < 0$ indicates that the preferred traversal decision is favored. The ranking loss is obtained by applying a non-linear activation function σ to Δ

$$\mathcal{L}_{\text{rank}} = \sigma(\Delta).$$

This work evaluates ReLU and GELU as activation functions. An additional regularization term encourages the embedding norms to remain close to one

$$\mathcal{L}_{\text{reg}} = \sum_{x \in \{c, e, p, n\}} (\|\phi(x)\|_2 - 1)^2.$$

For a single embedding dimension, the resulting loss is

$$\mathcal{L} = \mathcal{L}_{\text{rank}} + \mathcal{L}_{\text{reg}}.$$

The objective therefore optimizes embedding distances for causal graph traversal rather than semantic similarity.

5.3 Matryoshka Representation Learning

To support efficient causal knowledge graph traversal at different embedding dimensions, the ranking objective is extended using Matryoshka Representation Learning [13]. Rather than optimizing only the full embedding vector, the objective is applied to multiple nested prefixes of each representation. Let D_{model} denote the native embedding dimensionality of the underlying model. The set of training dimensions is defined as

$$\mathcal{M} = \{D_{\text{model}}\} \cup \{2^k \mid k \in \mathbb{N}, 2 \leq 2^k < D_{\text{model}}\}.$$

The dimension 768 is additionally included in $\mathcal{M}$ to enable direct comparison with MPNet and Granite, whose native dimensionality is 768 [29, 31].

$$\mathcal{M} \leftarrow \mathcal{M} \cup \{768\}.$$

For each $m \in \mathcal{M}$, let

$$\phi_m(x) = \phi(x)_{1:m}$$

denote the first m components of the embedding of concept x . Let f_m , g_m , and h_m denote the corresponding A^* evaluation, path-cost, and heuristic terms computed using ϕ_m . The corresponding evaluation difference is

$$\begin{aligned} \Delta_m &= f_m(p) - f_m(n) \\ &= [g_m(p) + h_m(p)] - [g_m(n) + h_m(n)] \\ &= [d(\phi_m(c), \phi_m(p)) + d(\phi_m(p), \phi_m(e))] \\ &\quad - [d(\phi_m(c), \phi_m(n)) + d(\phi_m(n), \phi_m(e))]. \end{aligned}$$

The dimension-specific ranking loss is

$$\mathcal{L}_{\text{rank}}^{(m)} = \sigma(\Delta_m).$$

The overall training loss is

$$\mathcal{L}_{\text{total}} = \sum_{m \in \mathcal{M}} \mathcal{L}_{\text{rank}}^{(m)} + \mathcal{L}_{\text{reg}}.$$

Thus, every Matryoshka prefix contributes a ranking loss, while the norm regularization remains global. Applying the objective at multiple prefix lengths encourages information relevant to causal graph traversal to be concentrated in the earlier embedding dimensions. This enables a single fine-tuned model to support multiple embedding dimensions without additional training, allowing the representation size to be selected according to the desired trade-off between computational efficiency and heuristic quality.

5.4 Validation Procedure

Although the training objective optimizes local ranking decisions, validation measures whether the learned embedding space improves global graph traversal. At the beginning of each validation epoch, embeddings for all graph nodes are recomputed using the current state of the model and stored in a temporary lookup cache. For each distinct pair (c, e) occurring in the supervision dataset $\mathcal{D}_{\text{val}}^d$ within a validation batch, an A^* search is performed from the current node c to the target node e using these updated embeddings and distance metric d . The number of nodes visited before reaching the target is recorded as $v(c, e)$. Let P_{val}^d denote the collection of evaluated pairs after batch-wise deduplication. The primary validation metric is the average number of visited nodes

$$\text{AvgVisited} = \frac{1}{|P_{\text{val}}^d|} \sum_{(c,e) \in P_{\text{val}}^d} v(c, e).$$

Lower values indicate that fewer graph nodes are visited on average before reaching the target and therefore correspond to more efficient traversal.

5.5 Hyperparameter Optimization

Before final training, model-specific hyperparameter optimization is performed with Optuna [35]. The search jointly optimizes the learning rate, activation function (ReLU or GELU), and distance metric (cosine or Euclidean) using the Tree-structured Parzen Estimator sampler [41]. The learning rate is sampled logarithmically from $[2.5 \times 10^{-5}, 3 \times 10^{-5}]$. This interval was selected based on preliminary experiments indicating stable and competitive validation performance within this range. Each trial performs a short training run and is evaluated by the average number of nodes visited during validation A^* searches. Early stopping terminates training when this metric no longer improves [42], while Optuna’s MedianPruner stops trials that underperform the median result of previously completed trials at the same training step [43]. For

each embedding model, hyperparameter optimization first runs 30 completed or pruned trials and then continues one trial at a time. The search stops after five consecutive trials without a lower average number of visited nodes on validation. An improvement resets this counter, allowing up to 50 trials. Each trial runs for at most 10 epochs with an early-stopping patience of 3 validation epochs. Training uses the AdamW optimizer [44] and a ReduceLROnPlateau scheduler [45], which halves the learning rate after one validation epoch without improvement. The effective batch size is fixed at 128, using gradient accumulation for smaller physical batches [46, 47].

5.6 Final Model Training

After hyperparameter optimization, the configuration with the lowest average number of visited nodes is selected for each embedding model. Final training uses the selected learning rate, activation function, and distance metric for up to 50 epochs with early stopping and a patience of 10 validation epochs. The checkpoint with the lowest average number of visited nodes on the validation set is retained as the final model and saved together with its training metadata.

5.7 Final Embedding Precomputation

After training, embeddings are precomputed for all graph-node concepts required during evaluation and inference. The node concepts are stored in a fixed order in a shared JSONL file. All embedding matrices follow this same ordering, allowing each node concept to be mapped to its row index and its embedding to be retrieved directly from the corresponding memory-mapped NumPy matrix [38]. The model first produces embeddings at its native dimensionality. For every Matryoshka dimension used in the experiments, the corresponding prefix of each full embedding is stored in a separate matrix. During graph traversal, A^* loads the matrix for the selected model and dimension instead of computing embeddings with the encoder at runtime. This reduces inference time and enables direct indexed access to all node embeddings. The approach therefore trades additional disk space for lower runtime.

Chapter 6

Results

This chapter evaluates the proposed embedding-guided A^* search for binary causal question answering, focusing on both effectiveness and search efficiency. The evaluation addresses four research questions. RQ1 assesses the impact of fine-tuning on the embedding heuristic, while RQ2 analyzes the role of Matryoshka dimensionality. RQ3 compares the selected A^* configuration with the BFS and RL baselines. Finally, RQ4 evaluates its generalization across different datasets and causal graph variants.

Section 6.1 introduces the experimental setup and evaluation metrics. The subsequent analyses examine the effect of fine-tuning in Section 6.2 and Matryoshka dimensionality in Section 6.3, addressing RQ1 and RQ2, respectively. Section 6.4 presents the final comparison with the baselines and evaluates performance across datasets and graph variants, addressing RQ3 and RQ4. Finally, Section 6.5 concludes the analysis with an ablation study of the activation function and distance metric.

6.1 Evaluation Setup

This section describes the evaluation procedure, datasets, causal graphs, models and baselines, and the methodology used throughout the experiments.

6.1.1 Evaluation Procedure

The evaluation is organized in two stages. First, MS MARCO validation [1, 48] with CauseNet Precision [5] is used to analyze fine-tuning and Matryoshka dimensionality and select the final A^* configuration. Second, the selected configuration is evaluated on the MS MARCO and SemEval [1, 49, 50] test splits across CauseNet Precision, CauseNet Full [5], and CEG Filtered [7], and compared with BFS and the RL approach of Blübaum and Heindorf [1].

Table 6.1: Number of causal questions in MS MARCO and SemEval. “Total” reports all causal samples in each split, while the graph columns report the samples retained after graph-node coverage filtering, where both the potential cause and effect are covered by the corresponding graph.

Split	Total		CauseNet _{prec}		CauseNet _{Full}		CEG _{Filtered}	
	Pos.	Neg.	Pos.	Neg.	Pos.	Neg.	Pos.	Neg.
MS MARCO [1, 48]								
Train	1837	0	1213	0	1618	0	389	0
Validation	194	47	137	22	176	35	44	8
Test	223	40	143	20	189	31	41	7
SemEval [1, 49, 50]								
Train	694	690	541	177	680	615	496	416
Validation	84	89	66	28	81	77	57	65
Test	87	86	66	24	84	76	57	53

6.1.2 Datasets

The causal question answering datasets used in this thesis are derived subsets rather than the complete original datasets. The MS MARCO data are based on causal questions extracted from the original MS MARCO dataset [48] as part of Webis-CausalQA-22 [16], while the SemEval data are based on the causal-relation subset of SemEval-2010 Task 8 [49] introduced by Sharp et al. [50]. The binary causal splits for both datasets were provided by Blübaum and Heindorf [1]. During evaluation, examples are skipped if the potential cause, the effect, or both are absent from the evaluated graph. Table 6.1 reports the dataset sizes and class distributions in the original splits and after graph-node coverage filtering. Because graph-node coverage differs across graphs, each graph is evaluated on a different subset of questions with a different class distribution. Differences in effectiveness across graphs should therefore not be attributed solely to the graph itself.

6.1.3 Causal Graphs

The evaluation uses three graph settings. CauseNet Precision is the high-precision subset of CauseNet [5] and is used for training, validation, model selection, and the main comparison to prior work. CauseNet Full is used to test scalability to a substantially larger graph. The third evaluated graph is a filtered version of the original CEG [7]. Because CEG is very dense and contains many low-confidence edges, filtering is applied using the causal-strength formulation of Luo et al. [51], which combines necessity and sufficiency scores.

Graph	Nodes	Edges
CauseNet Precision	80,211	197,370
CauseNet Full	12,185,890	11,606,945
CEG Filtered	77,264	21,507,177
CEG	79,865	92,270,736

Table 6.2: Number of nodes and unique directed edges after graph preprocessing. CEG is reported for reference, while CEG Filtered is used in the evaluation.

For each CEG edge from cause c to effect e , the causal strength is computed as

$$\text{CS}(c, e) = \text{necessity}(c, e)^\lambda \cdot \text{sufficiency}(c, e)^{1-\lambda},$$

where $\lambda \in [0, 1]$ is a weighting parameter. This thesis uses $\lambda = 0.9$, following Luo et al., and retains only edges with $\text{CS}(c, e) \geq 0.1$. All graph files undergo the same preprocessing, which removes empty or punctuation-only concepts and self-loops and normalizes labels by replacing underscores with spaces and stripping surrounding whitespace. Table 6.2 reports the resulting graph sizes.¹

6.1.4 Models and Baselines

For embedding-guided A^* , five embedding backbones are evaluated alongside the RL baseline of Blübaum and Heindorf [1]. The A^* models and RL policy are trained using the MS MARCO training split and CauseNet Precision. Unlike the original RL setup, the policy is trained and evaluated without inverse edges because these may introduce transitions that do not represent causality in the queried direction. This ensures that all methods operate on the same forward directed-reachability task. Table 6.3 defines the model identifiers, parameter counts, embedding dimensions, and concise labels used throughout the results. Table 6.4 reports the selected hyperparameters and final training epoch for each fine-tuned embedding model. The epoch denotes the zero-indexed last validation improvement before early stopping with a patience of 10. The checkpoint from this epoch is retained as the final model.

¹The publicly available CEG release, in which cause–effect pairs with a reported frequency of 5 or lower are already excluded, contains approximately 92.3 million edges. In contrast, the official CEG release documentation [7] reports approximately 89.1 million cause–effect pairs for this version. The available release information is insufficient to determine the source of this discrepancy conclusively. The official CauseNet statistics report 199,806 relations and 80,223 concepts for CauseNet Precision and 11,609,890 relations and 12,186,195 concepts for CauseNet Full [5]. The slightly smaller counts in Table 6.2 result from the preprocessing used in this thesis, which removes empty or punctuation-only concepts and self-loops and normalizes concept labels.

Table 6.3: Models used in the evaluation. The table reports the model labels used throughout this thesis, the corresponding model identifiers, parameter counts, and native embedding dimensions D .

Model label	Model identifier	Params.	D
RL [1]	LSTM policy model	12M	–
BGE [30, 52]	BAAI/bge-large-en-v1.5	335M	1024
Granite [31, 53]	ibm-granite/granite-embedding-english-r2	149M	768
MPNet [29, 54]	sentence-transformers/all-mpnet-base-v2	109M	768
mxbai [32, 55]	mixedbread-ai/mxbai-embed-large-v1	335M	1024
Qwen [33, 56]	Qwen/Qwen3-Embedding-0.6B	600M	1024

(a) Selected hyperparameters				(b) Selected epoch	
Model	Activation	Metric	lr [10^{-5}]	Model	Epoch
MPNet	ReLU	Cosine	2.673	MPNet	0
BGE	ReLU	Cosine	2.924	BGE	0
mxbai	ReLU	Cosine	2.935	mxbai	3
Granite	ReLU	Euclidean	2.709	Granite	10
Qwen	ReLU	Cosine	2.887	Qwen	10

Table 6.4: Final training settings. Panel (a) reports the selected activation function, distance metric, and learning rate. Panel (b) reports the zero-indexed epoch of the last improvement in validation A^* cost. Its checkpoint is retained for evaluation.

6.1.5 Evaluation Methodology

This section describes the experimental evaluation, including search budgets, metrics, runtime measurement, model selection, and significance testing.

Search Budgets

The evaluation reports two BFS variants and budgeted A^* . BFS (uncapped) runs without a visit limit and serves as the exhaustive reachability baseline. For BFS (capped) and A^* , budgets are set to the 95th percentile (p_{95}) of visited nodes from successful searches on the MS MARCO training split with CauseNet Precision. BFS uses one global cap, whereas A^* uses separate budgets for each embedding model and Matryoshka dimension, as reported in Table 6.5. Uncapped A^* is additionally reported in the fine-tuning analysis to separate the effect of fine-tuning from the visit budget. The Matryoshka and final evaluations use the budgeted variants. Following the implementation of Blübaum and Heindorf [1], RL has no visit budget and instead uses beam search with a beam width of 50, a STAY action, and a rollout length of two.

Table 6.5: A^* visit budgets τ derived from the p_{95} of visited nodes in successful searches on MS MARCO train with CauseNet Precision. Pretrained models are reported at their full embedding dimension, while fine-tuned models are reported across all evaluated Matryoshka dimensions.

Model	Dimension										
	2	4	8	16	32	64	128	256	512	768	1024
BGE	–	–	–	–	–	–	–	–	–	–	379
BGE tuned	682	96	27	12	10	8	8	8	9	10	10
Granite	–	–	–	–	–	–	–	–	–	825	–
Granite tuned	68	16	12	13	16	23	25	23	29	32	–
MPNet	–	–	–	–	–	–	–	–	–	503	–
MPNet tuned	1,120	178	41	18	13	14	15	17	21	22	–
mxbai	–	–	–	–	–	–	–	–	–	–	376
mxbai tuned	247	62	20	12	9	8	9	9	9	9	9
Qwen	–	–	–	–	–	–	–	–	–	–	594
Qwen tuned	253	105	10	6	5	5	5	5	5	6	5

Metrics

Effectiveness is measured using precision, recall, and F_1 . Efficiency is measured using the average number of visited nodes and the average runtime per query in milliseconds. For BFS and A^* , visited nodes count nodes processed during graph search, whereas for RL, following the original implementation, they count unique nodes in retained beam paths until the target is first encountered. Runtime additionally reflects neural inference, distance computation, and implementation overhead, whereas visited nodes provide a hardware-independent measure of traversal effort that is less sensitive to system load and hardware variation. The two metrics therefore capture complementary aspects of algorithmic search effort and practical execution cost.

Runtime Measurement

All evaluations were run on the Webis Slurm infrastructure using eight CPU cores, 128 GB of memory, and one H100 GPU.² Embedding computations use CUDA, while BFS runs on the CPU. Runtime is measured per query using Python’s `time.perf_counter()`, with CUDA synchronization before and after traversal. One untimed warm-up is performed per strategy. Graph loading, embedding precomputation, and cache loading are excluded, and reported runtimes are averages over a single evaluation run.

²Further details are available at <https://webis.de/facilities.html>.

Model Selection

The final A^* configuration is selected from the capped fine-tuned validation results across all Matryoshka dimensions by balancing effectiveness and search efficiency. First, candidates dominated by another configuration with at least as high an F_1 score and no more visited nodes N are removed. Among the remaining candidates, a knee is selected, representing a point beyond which improving one objective requires a disproportionate deterioration in the other [57]. The selection adapts the normalized difference-curve construction of Satopaa et al. [58]. F_1 and the logarithm of N are min-max normalized to $\widetilde{F}_1$ and $\widetilde{\log N}$, respectively. The score $(\widetilde{F}_1 - \widetilde{\log N})/\sqrt{2}$ measures the signed perpendicular distance from the equal-trade-off diagonal, with larger values indicating a better combination of high F_1 and low search effort. The configuration with the largest score is selected. Runtime, the visit budget, and the embedding dimension are used only as tie-breakers, in that order.

Statistical Significance Testing

Statistical significance is tested only for the final test comparison. For each dataset-graph combination, the selected A^* configuration is compared with BFS (uncapped), BFS (capped), and RL using a paired stratified bootstrap test [59, 60]. Positive and negative examples are resampled separately over 10,000 bootstrap samples. Bonferroni correction is applied to the three comparisons at a family-wise significance level of $\alpha = 0.05$. The reported p-values are Bonferroni-adjusted and compared with 0.05, whereas the 95% confidence intervals remain unadjusted for each individual comparison.

6.2 Effect of Fine-Tuning

RQ1 asks whether fine-tuning improves the embedding heuristic used by A^* . Table 6.6 compares each model before and after fine-tuning at its full embedding dimension on MS MARCO validation with CauseNet Precision, using the best hyperparameter configuration from Table 6.4. The BFS variants and RL are included as references. RL appears only in the uncapped group because it has no visit budget, although its beam width and rollout length remain fixed. The capped runs use their respective p_{95} budgets from MS MARCO train, whereas the uncapped runs remove the visit limit.

With the cap, pretrained embeddings reach F_1 scores between 86.2% and 87.1% while visiting 66.1 to 142.0 nodes on average. Fine-tuning reduces average visited nodes to between 3.5 and 11.0 and runtime for every model, but

also lowers F_1 to between 74.6% and 82.4%. Without the cap, all A^* variants obtain the same F_1 of 90.6%, as uncapped A^* is complete. Fine-tuning nevertheless reduces visited nodes for all five models and runtime for BGE, Granite, MPNet, and Qwen. For mxbai, runtime increases slightly despite visiting fewer nodes on average, as a few unusually expensive searches raise the mean runtime. Thus, fine-tuning and visit budgets substantially improve search efficiency, while capped search trades some effectiveness for fewer visited nodes by terminating searches that exceed the budget.

Because the fine-tuned models use different p_{95} -derived budgets τ , Figure 6.1 compares them at full dimensionality over a shared τ grid. MPNet provides the strongest trade-off across most of the range, followed by Granite and Qwen. mxbai requires more search effort for comparable F_1 , while BGE remains least effective. Larger budgets generally improve effectiveness by allowing A^* to explore more nodes, but with diminishing returns.

Table 6.6: Validation results on MS MARCO with CauseNet Precision at each model’s full embedding dimension. Pretrained (PT) and fine-tuned (FT) A^* are compared with BFS and RL under capped and uncapped search. Capped runs use their p_{95} budget τ . F_1 is reported in percent, N denotes average visited nodes, and T denotes runtime in milliseconds per query. The best value in each metric and search regime is bold, including ties.

Model	Variant	τ	Capped			Uncapped		
			F_1	N	T	F_1	N	T
BFS		1,316	86.6	184.8	0.7	90.6	1,659.8	2.2
RL		–	–	–	–	58.3	37.5	73.4
BGE	PT	379	86.6	68.6	9.1	90.6	1,342.4	41.5
	FT	10	79.8	5.2	1.0	90.6	1,096.4	33.3
Granite	PT	825	86.2	142.0	13.2	90.6	1,448.4	43.0
	FT	32	82.3	11.0	1.9	90.6	1,252.8	29.8
MPNet	PT	503	86.6	85.1	9.9	90.6	1,394.9	42.1
	FT	22	82.4	7.5	1.7	90.6	1,101.4	34.2
mxbai	PT	376	86.6	66.1	8.0	90.6	1,323.7	38.1
	FT	9	74.6	5.2	0.9	90.6	1,259.3	40.1
Qwen	PT	594	87.1	105.4	11.5	90.6	1,407.8	44.0
	FT	5	78.4	3.5	0.6	90.6	1,140.1	38.1

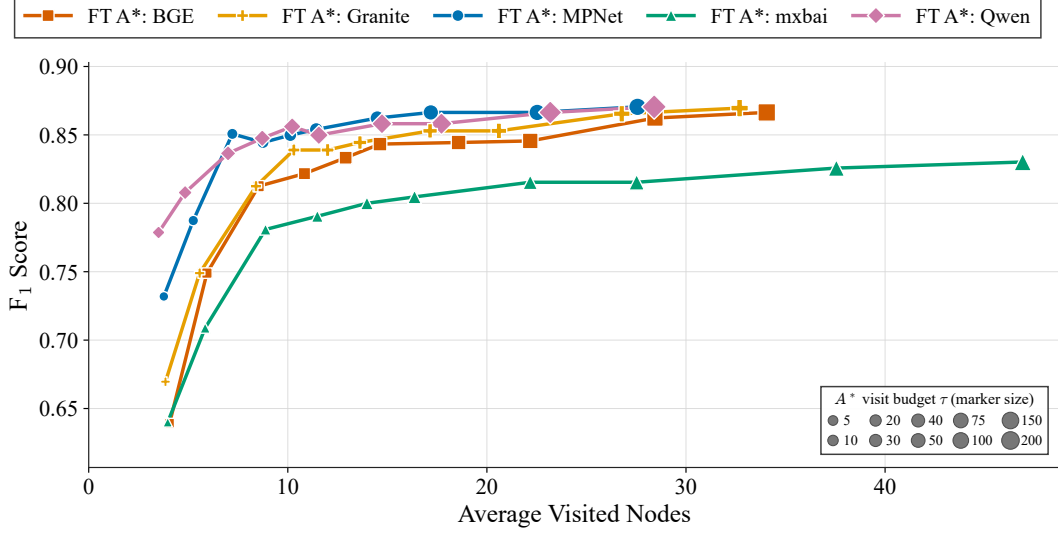


Figure 6.1: Efficiency–effectiveness trade-off for the five full-dimensional fine-tuned embedding models on MS MARCO validation with CauseNet Precision. F_1 score is shown against the measured average number of visited nodes, with marker size indicating the corresponding A^* visit budget τ .

6.3 Effect of Matryoshka Dimensionality

RQ2 examines the trade-off between effectiveness and efficiency across Matryoshka embedding dimensions. Figure 6.2 reports only capped fine-tuned models, using the same p_{95} -budget procedure across dimensions. Lower dimensions often reduce the visit budget, search effort, and runtime, although the effect is model-dependent and non-monotonic. For Granite, reducing the full 768-dimensional embedding to 32 dimensions lowers τ from 32 to 16, visited nodes from 11.0 to 7.1, and runtime from 1.9 to 1.2 ms, while F_1 changes only slightly from 82.3% to 81.9%. At 64 dimensions, F_1 instead improves to 83.5%, with $\tau = 23$, 9.1 visited nodes, and a runtime of 1.5 ms. MPNet shows a similar intermediate optimum. At 64 dimensions, it retains the full model’s 82.4% F_1 while reducing τ from 22 to 14, visited nodes from 7.5 to 6.1, and runtime from 1.7 to 1.3 ms. Its two- and four-dimensional prefixes achieve higher F_1 scores of 86.5% and 84.8%, but require 223.8 and 45.8 visited nodes, showing that lower dimensionality does not necessarily yield more efficient search. This instability at very low dimensions also appears for BGE and mxbai. Their two-dimensional variants reach 83.6% and 79.2% F_1 , respectively, but require 164.2 and 78.2 visited nodes. Qwen lies at the efficiency-oriented end of the trade-off, requiring only 3.8 visited nodes and 0.7 ms at 768 dimensions, but reaching 79.4% F_1 . Different prefixes alter the embedding distances and there-

fore the order in which A^* explores nodes. This helps explain why reducing dimensionality does not consistently reduce search effort. Overall, intermediate dimensions can provide better operating points than either the full embedding or very short prefixes while retaining sufficient information for effective search.

Among these operating points, MPNet-64 is the closest efficiency-oriented alternative to Granite-64, saving 2.9 node visits and 0.24 ms while losing 1.1 percentage points of F_1 . On the global frontier, Granite-64 consequently obtains the higher Pareto-knee score, 0.285 compared with 0.257 for MPNet-64. Under the balanced effectiveness–efficiency objective, this effectiveness gain outweighs the modest additional search cost. MPNet-64, or Qwen-768 under a stricter latency constraint, could nevertheless be preferable when efficiency is prioritized more strongly. For the stated objective, the fine-tuned Granite-64 configuration with $\tau = 23$ provides the best overall operating point and is therefore selected for the final test evaluation.

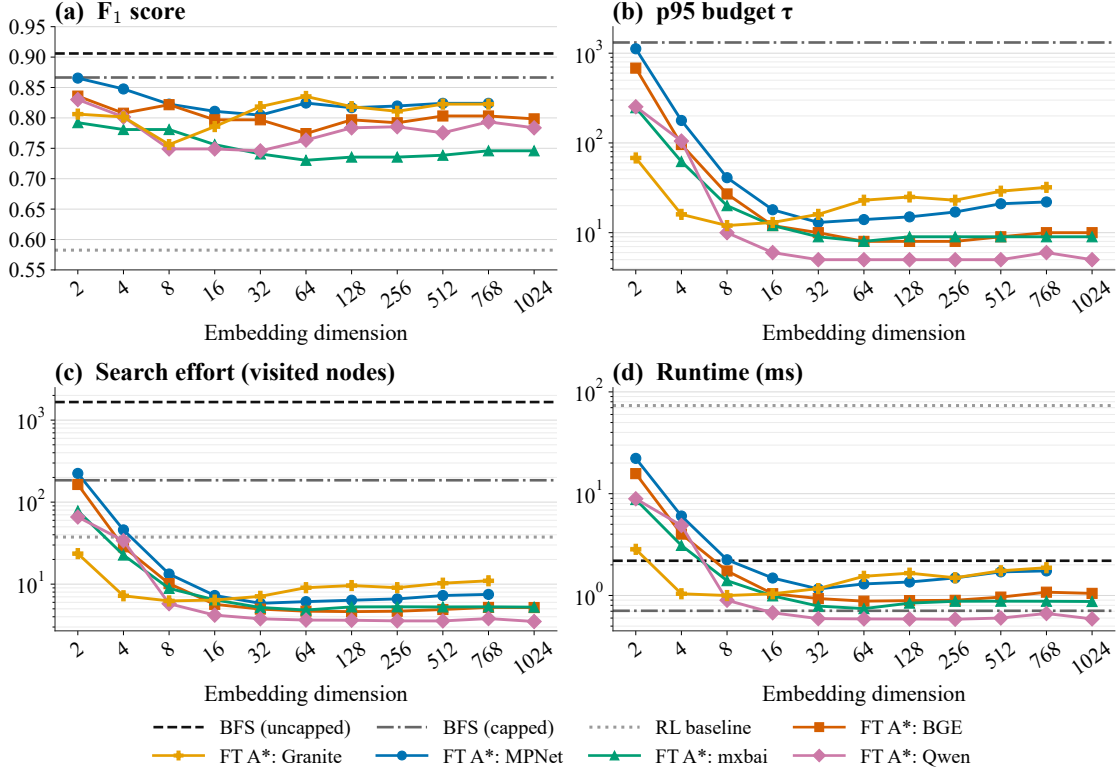


Figure 6.2: Validation performance of capped fine-tuned models on MS MARCO with CauseNet Precision across Matryoshka embedding dimensions. Shown are F_1 , the p_{95} visit budget τ , average visited nodes, and runtime. The budget, visited nodes, and runtime use logarithmic vertical axes.

6.4 Final Test Results

RQ3 compares the selected A^* configuration with the BFS and RL baselines, while RQ4 examines its generalization across datasets and causal graph variants. Table 6.10 reports the results on MS MARCO and SemEval for CauseNet Precision, CauseNet Full, and CEG Filtered. A^* achieves a higher F_1 score, visits fewer nodes, and has a lower runtime than RL in all six settings. The comparison with BFS instead reveals an effectiveness–efficiency trade-off. On the smaller CauseNet Precision graph, BFS achieves the highest F_1 score on both datasets, whereas A^* visits only 6.5 nodes on MS MARCO and 11.4 on SemEval. Capped BFS is faster in these settings, but visits 20.6 and 38.3 times as many nodes, respectively. This suggests that on smaller graphs, the overhead of embedding guidance can outweigh its savings in graph exploration.

On the larger CauseNet Full and denser CEG Filtered, the benefits of guided search become more pronounced. On CauseNet Full, A^* visits only 7.5 nodes on MS MARCO and 14.2 on SemEval, compared with 30,099.3 and 160,715.3 for uncapped BFS. On MS MARCO, this substantial reduction in search effort comes with a lower F_1 score than either BFS variant. On SemEval, however, A^* exceeds uncapped BFS by 9.5 percentage points and trails capped BFS by only 0.6 points. On CEG Filtered, A^* attains an F_1 score of 92.0% on MS MARCO, matching capped BFS and slightly exceeding uncapped BFS while visiting only 2.3 nodes on average. Its F_1 score of 68.5% on SemEval lies between those of the two BFS variants, again with substantially fewer visited nodes and a lower runtime. These results demonstrate generalization beyond the CauseNet Precision–MS MARCO validation setting, with embedding guidance becoming more beneficial as graph exploration grows more expensive.

Tables 6.7–6.9 report paired stratified bootstrap tests for F_1 score differences between A^* and each baseline. Positive Δ favors A^* and negative Δ the baseline, with differences in percentage points. The 95% confidence intervals are unadjusted, while $p_{\text{Bonf.}}$ denotes the Bonferroni-adjusted p -value. The tests show that effectiveness depends on the graph and dataset. On MS MARCO, A^* performs significantly worse than both BFS variants on CauseNet Precision and CauseNet Full, whereas neither comparison is significant on CEG Filtered. On SemEval, the pattern changes. A^* significantly outperforms uncapped BFS on CauseNet Full, while its differences from both BFS variants on CauseNet Precision and capped BFS on CauseNet Full are not significant. The advantage over RL is more consistent, reaching significance on both datasets for both CauseNet variants, but not on CEG Filtered. Overall, the efficiency gains of A^* do not imply a uniform loss in effectiveness. This is most apparent on CauseNet Full, where A^* reduces graph exploration by several orders of magnitude while remaining competitive with BFS and outperforming RL.

Table 6.7: Paired stratified bootstrap significance tests on CauseNet Precision.

Dataset	Baseline	Δ	95% CI	$p_{\text{Bonf.}}$	Sig.
MS MARCO	BFS (uncapped)	-3.9	$[-6.7, -1.6]$	0.0123	yes
MS MARCO	BFS (capped)	-3.5	$[-6.2, -1.1]$	0.0306	yes
MS MARCO	RL	+16.4	$[+10.9, +22.5]$	0.0003	yes
SemEval	BFS (uncapped)	-5.2	$[-11.1, -0.2]$	0.1839	no
SemEval	BFS (capped)	-4.9	$[-10.0, -0.8]$	0.1209	no
SemEval	RL	+16.0	$[+7.0, +26.1]$	0.0048	yes

Table 6.8: Paired stratified bootstrap significance tests on CauseNet Full.

Dataset	Baseline	Δ	95% CI	$p_{\text{Bonf.}}$	Sig.
MS MARCO	BFS (uncapped)	-7.1	$[-10.6, -4.0]$	0.0006	yes
MS MARCO	BFS (capped)	-4.3	$[-7.2, -1.8]$	0.0084	yes
MS MARCO	RL	+13.0	$[+8.3, +17.9]$	0.0003	yes
SemEval	BFS (uncapped)	+9.5	$[+2.5, +15.6]$	0.0171	yes
SemEval	BFS (capped)	-0.6	$[-6.9, +5.0]$	1.0000	no
SemEval	RL	+24.8	$[+14.5, +35.5]$	0.0003	yes

Table 6.9: Paired stratified bootstrap significance tests on CEG Filtered.

Dataset	Baseline	Δ	95% CI	$p_{\text{Bonf.}}$	Sig.
MS MARCO	BFS (uncapped)	+1.0	$[+0.0, +3.2]$	1.0000	no
MS MARCO	BFS (capped)	+0.0	$[+0.0, +0.0]$	1.0000	no
MS MARCO	RL	+7.3	$[+0.6, +15.7]$	0.1671	no
SemEval	BFS (uncapped)	-0.2	$[-5.5, +4.5]$	1.0000	no
SemEval	BFS (capped)	+0.2	$[-4.6, +4.6]$	1.0000	no
SemEval	RL	+8.3	$[-2.2, +20.0]$	0.4260	no

6.5 Activation and Distance Ablation

This section analyzes the effects of the activation function and distance metric during fine-tuning. The same Granite model is trained with all four combinations of the ReLU or GELU activation function and Euclidean or cosine distance, while keeping all other settings fixed. The ablation is evaluated on CauseNet Precision, CauseNet Full, and CEG Filtered with both test datasets, using the selected 64-dimensional Matryoshka prefix and A^* budget $\tau = 23$.

Table 6.10: Final test results for the selected fine-tuned Granite-64 A^* configuration with $\tau = 23$, compared with uncapped BFS (BFS-U), capped BFS (BFS-C), and the reinforcement learning baseline. CauseNet Prec. denotes CauseNet Precision. Precision (P), recall (R), and F_1 are reported in percent. T denotes runtime in milliseconds per query and N the average number of visited nodes. Asterisks mark F_1 scores that differ significantly from the selected A^* configuration within each dataset–graph setting using a paired stratified bootstrap test with Bonferroni correction. Bold indicates the best value in each metric and dataset–graph setting, including ties.

Graph	Approach	MS MARCO					SemEval				
		P	R	F_1	T	N	P	R	F_1	T	N
CauseNet Prec.	BFS-U	87.1	94.4	90.6*	1.4	1157.3	89.2	87.9	88.5	6.6	6332.2
CauseNet Prec.	BFS-C	87.5	93.0	90.2*	0.5	134.9	91.8	84.8	88.2	1.2	436.7
CauseNet Prec.	RL	89.2	58.0	70.3*	67.7	32.0	97.1	51.5	67.3*	114.4	38.3
CauseNet Prec.	A^*	86.7	86.7	86.7	1.1	6.5	92.6	75.8	83.3	1.8	11.4
CauseNet Full	BFS-U	85.6	94.7	89.9*	59.5	30099.3	56.3	95.2	70.8*	294.1	160715.3
CauseNet Full	BFS-C	86.1	88.4	87.2*	9.4	135.1	73.1	90.5	80.9	28.4	450.8
CauseNet Full	RL	88.6	57.7	69.9*	658.1	44.6	94.3	39.3	55.5*	885.1	52.7
CauseNet Full	A^*	86.7	79.4	82.9	7.2	7.5	86.3	75.0	80.3	15.6	14.2
CEG Filtered	BFS-U	85.1	97.6	90.9	44.3	492.1	52.8	98.2	68.7	45.5	776.2
CEG Filtered	BFS-C	87.0	97.6	92.0	5.8	30.2	52.9	96.5	68.3	9.1	42.4
CEG Filtered	RL	89.2	80.5	84.6	5129.0	24.3	77.8	49.1	60.2	5131.9	54.7
CEG Filtered	A^*	87.0	97.6	92.0	0.8	2.3	56.2	87.7	68.5	4.4	7.2

Table 6.11 reports the complete ablation results, while Figure 6.3 summarizes the relationship between F_1 and search effort. ReLU–Euclidean provides the strongest overall trade-off across the evaluated settings. On both CauseNet variants, it achieves the highest F_1 and the lowest number of visited nodes. The results on CEG Filtered are more mixed. With MS MARCO, ReLU–Euclidean achieves the lowest search effort while tying for the highest F_1 . With SemEval, GELU–Cosine achieves the highest F_1 , whereas ReLU–Euclidean requires the fewest visited nodes. These results indicate that the preferred activation and distance combination can vary across graphs and datasets, although ReLU–Euclidean is the most consistent across settings.

Small runtime differences from Table 6.10 result from separate evaluation runs. Among the ablation configurations, GELU is fastest in four of the six settings, including both CauseNet Full evaluations, but these gains generally coincide with lower F_1 and more visited nodes. For example, GELU–Cosine reduces runtime from 14.9 to 7.4 ms on CauseNet Full with SemEval compared with ReLU–Euclidean, but loses 4.3 F_1 points and visits 1.2 more nodes. Overall, ReLU–Euclidean provides the most consistent effectiveness–efficiency trade-off despite the runtime advantages of GELU in some settings.

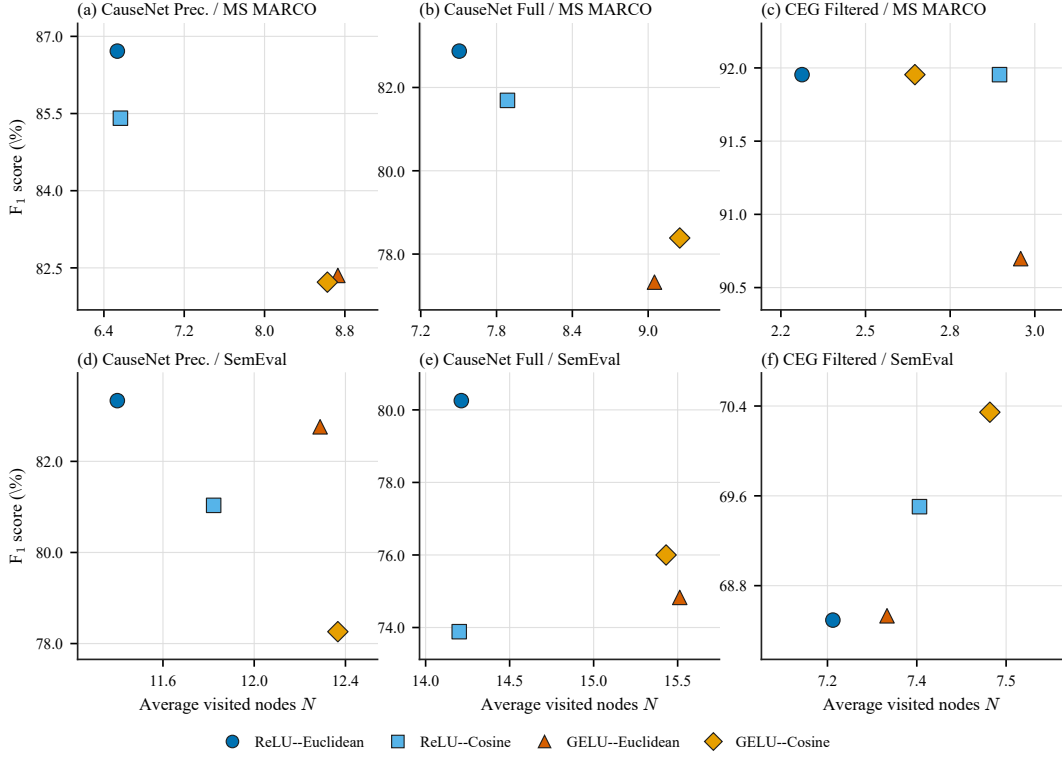


Figure 6.3: Effectiveness–efficiency trade-off across activation and distance configurations. F_1 is plotted against average visited nodes across all six test settings.

Table 6.11: Activation and distance ablation for Granite-64 across all test settings with $\tau = 23$. Bold indicates the best value per metric and setting, including ties.

Graph	Configuration	MS MARCO					SemEval				
		P	R	F_1	T	N	P	R	F_1	T	N
CauseNet Prec.	ReLU–Euclidean	86.7	86.7	86.7	1.0	6.5	92.6	75.8	83.3	1.8	11.4
CauseNet Prec.	ReLU–Cosine	87.0	83.9	85.4	1.1	6.6	94.0	71.2	81.0	1.9	11.8
CauseNet Prec.	GELU–Euclidean	86.8	78.3	82.4	1.3	8.7	96.0	72.7	82.8	1.4	12.3
CauseNet Prec.	GELU–Cosine	87.4	77.6	82.2	1.3	8.6	91.8	68.2	78.3	1.5	12.4
CauseNet Full	ReLU–Euclidean	86.7	79.4	82.9	6.9	7.5	86.3	75.0	80.3	14.9	14.2
CauseNet Full	ReLU–Cosine	87.3	76.7	81.7	6.5	7.9	79.5	69.0	73.9	12.3	14.2
CauseNet Full	GELU–Euclidean	85.8	70.4	77.3	5.9	9.1	87.3	65.5	74.8	7.6	15.5
CauseNet Full	GELU–Cosine	86.1	72.0	78.4	4.8	9.3	86.4	67.9	76.0	7.4	15.4
CEG Filtered	ReLU–Euclidean	87.0	97.6	92.0	0.8	2.3	56.2	87.7	68.5	4.4	7.2
CEG Filtered	ReLU–Cosine	87.0	97.6	92.0	1.4	2.9	58.3	86.0	69.5	4.4	7.4
CEG Filtered	GELU–Euclidean	86.7	95.1	90.7	1.5	3.0	57.0	86.0	68.5	4.0	7.3
CEG Filtered	GELU–Cosine	87.0	97.6	92.0	1.4	2.6	58.0	89.5	70.3	5.1	7.5

Chapter 7

Discussion

The main advantage of embedding-guided A^* lies in its ability to reduce search effort. It avoids the exhaustive exploration of BFS by prioritizing promising nodes and avoids the repeated neural inference of RL by relying on precomputed node embeddings. This makes the approach particularly suitable for large or dense causal graphs, where exhaustive traversal becomes increasingly expensive. This efficiency, however, comes with a trade-off because bounded exploration may terminate before an existing path is discovered, reducing recall. At the same time, limiting exploration can reduce spurious positive answers that arise only after extensive graph traversal. The reliability of the answers also depends on the underlying causal knowledge graph. Automatically constructed resources such as CauseNet and CEG may contain missing, noisy, ambiguous, or overly general relations, so a discovered path represents a connection encoded in the graph rather than definitive evidence of a real-world causal relationship. In addition, the approach assumes that causal relations can be meaningfully composed along a directed path, although causality is not necessarily transitive. Finally, the method is limited to binary causal questions. It determines whether a directed path between the graph nodes corresponding to the cause and effect can be found, but does not assess the quality, plausibility, or strength of the discovered causal relation. It also does not estimate causal effect sizes, compare alternative causes, or support more complex forms of causal reasoning such as interventional and counterfactual queries. These limitations restrict the proposed approach to efficient causal path discovery rather than general causal inference.

Chapter 8

Conclusion

This thesis investigated whether a learned embedding-based heuristic can make A^* a practical approach for binary causal question answering over large causal knowledge graphs. Questions of the form “Does A cause B ?” were modeled as directed reachability problems, with fine-tuned embedding distances guiding graph traversal. The final configuration uses Granite Embedding R2 with a 64-dimensional Matryoshka prefix, ReLU activation, Euclidean distance, and a search budget of $\tau = 23$. The results show that embedding-guided A^* substantially reduces traversal cost while maintaining competitive predictive performance. Its efficiency advantage is particularly pronounced on large or dense graphs such as CauseNet Full and CEG Filtered. On these graphs, A^* is at least $1.3\times$ faster than capped BFS and $8.2\times$ faster than uncapped BFS, while reducing the number of visited nodes by factors of at least 5.9 and 107, respectively. It also outperforms the evaluated RL baseline in F_1 and runtime across all six final test settings. Overall, the findings demonstrate that learned embedding guidance provides an efficient alternative to exhaustive and policy-based traversal for binary causal graph search. Future work could address the limitations identified in this thesis. Adaptive, query-dependent search budgets could reduce recall loss caused by bounded exploration. Incorporating edge confidence, provenance, and path-quality estimation could improve robustness to noisy or ambiguous graph relations. Further work could also investigate the contextual validity of composed causal paths and extend the approach beyond binary reachability to tasks involving causal relation quality, effect strength, alternative causes, and interventional or counterfactual reasoning.

Bibliography

- [1] Lukas Blübaum and Stefan Heindorf. Causal question answering with reinforcement learning. In *Proceedings of the ACM Web Conference 2024*, pages 2204–2215. Association for Computing Machinery, 2024. doi: 10.1145/3589334.3645610. URL <https://doi.org/10.1145/3589334.3645610>.
- [2] Jonathan G. Richens, Ciarán M. Lee, and Saurabh Johri. Improving the accuracy of medical diagnosis with causal machine learning. *Nature Communications*, 11(1):3923, 2020. doi: 10.1038/s41467-020-17419-7. URL <https://doi.org/10.1038/s41467-020-17419-7>.
- [3] Matthew Blackwell. A framework for dynamic causal inference in political science. *American Journal of Political Science*, 57(2):504–520, 2013. doi: 10.1111/j.1540-5907.2012.00626.x. URL <https://doi.org/10.1111/j.1540-5907.2012.00626.x>.
- [4] Laura Andrea Barrero Guevara, Sarah C. Kramer, Tobias Kurth, and Matthieu Domenech de Cellès. Causal inference concepts can guide research into the effects of climate on infectious diseases. *Nature Ecology & Evolution*, 9(2):349–363, 2025. doi: 10.1038/s41559-024-02594-3. URL <https://doi.org/10.1038/s41559-024-02594-3>.
- [5] Stefan Heindorf, Yan Scholten, Henning Wachsmuth, Axel-Cyrille Ngonga Ngomo, and Martin Potthast. CauseNet: Towards a causality graph extracted from the web. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management, CIKM 2020*, pages 3023–3030. Association for Computing Machinery, 2020. doi: 10.1145/3340531.3412763. URL <https://doi.org/10.1145/3340531.3412763>.
- [6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022. ISBN 9780262046305. URL <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>.

- [7] Zhongyang Li, Xiao Ding, Ting Liu, J. Edward Hu, and Benjamin Van Durme. Guided generation of cause and effect. In Christian Bessiere, editor, *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, pages 3629–3636. International Joint Conferences on Artificial Intelligence Organization, July 2020. doi: 10.24963/ijcai.2020/502. URL <https://doi.org/10.24963/ijcai.2020/502>. Main track.
- [8] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136. URL <https://doi.org/10.1109/TSSC.1968.300136>.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- [10] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, EMNLP 2019*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL <https://doi.org/10.18653/v1/D19-1410>.
- [11] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. ISBN 9780521865715. URL <https://nlp.stanford.edu/IR-book/>.
- [12] Erich Schubert. A triangle inequality for cosine similarity. In Nora Reyes, Richard Connor, Nils Kriege, Daniyal Kazempour, Ilaria Bartolini, Erich Schubert, and Jian-Jia Chen, editors, *Similarity Search and Applications*, volume 13058 of *Lecture Notes in Computer Science*, pages 32–44, Cham, 2021. Springer. doi: 10.1007/978-3-030-89657-7_3. URL https://doi.org/10.1007/978-3-030-89657-7_3.
- [13] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 30233–30249. Curran Associates, Inc., 2022. URL

- https://proceedings.neurips.cc/paper_files/paper/2022/hash/c32319f4868da7613d78af9993100e42-Abstract-Conference.html.
- [14] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning, ICML 2010*, pages 807–814, 2010. URL <https://www.cs.toronto.edu/~hinton/absps/reluICML.pdf>.
- [15] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *CoRR*, abs/1606.08415, 2016. URL <https://arxiv.org/abs/1606.08415>.
- [16] Alexander Bondarenko, Magdalena Wolska, Stefan Heindorf, Lukas Blübaum, Axel-Cyrille Ngonga Ngomo, Benno Stein, Pavel Braslavski, Matthias Hagen, and Martin Potthast. CausalQA: A benchmark for causal question answering. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 3296–3308. International Committee on Computational Linguistics, 2022. URL <https://aclanthology.org/2022.coling-1.291/>.
- [17] Zeyu Wang. CausalBench: A comprehensive benchmark for evaluating causal reasoning capabilities of large language models. In *Proceedings of the 10th SIGHAN Workshop on Chinese Language Processing*, pages 143–151. Association for Computational Linguistics, 2024. URL <https://aclanthology.org/2024.sighan-1.17/>.
- [18] Emre Kıcıman, Robert Osazuwa Ness, Amit Sharma, and Chenhao Tan. Causal reasoning and large language models: Opening a new frontier for causality. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=mqoxLkX210>.
- [19] Sirui Chen, Mengying Xu, Kun Wang, Xingyu Zeng, Rui Zhao, Shengjie Zhao, and Chaochao Lu. CLEAR: Can language models really understand causal graphs? In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 6247–6265. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.363. URL <https://aclanthology.org/2024.findings-emnlp.363/>.
- [20] Martina Miliani, Serena Auriemma, Alessandro Bondielli, Emmanuele Chersoni, Lucia Passaro, Irene Sucameli, and Alessandro Lenci. ExpliCa: Evaluating explicit causal reasoning in large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 17335–17355. Association for Computational Linguistics, 2025. doi:

- 10.18653/v1/2025.findings-acl.891. URL <https://aclanthology.org/2025.findings-acl.891/>.
- [21] Haoang Chi, He Li, Wenjing Yang, Feng Liu, Long Lan, Xiaoguang Ren, Tongliang Liu, and Bo Han. Unveiling causal reasoning in large language models: Reality or mirage? In *Advances in Neural Information Processing Systems*, volume 37, pages 96640–96670. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/af2bb2b2280d36f8842e440b4e275152-Abstract-Conference.html.
- [22] Anpeng Wu, Kun Kuang, Minqin Zhu, Yingrong Wang, Yujia Zheng, Kairong Han, Baohong Li, Guangyi Chen, Fei Wu, and Kun Zhang. Causality for large language models. *CoRR*, abs/2410.15319, 2024. doi: 10.48550/arXiv.2410.15319. URL <https://arxiv.org/abs/2410.15319>.
- [23] Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014. doi: 10.1145/2629489. URL <https://doi.org/10.1145/2629489>.
- [24] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary G. Ives. Dbpedia: A nucleus for a web of open data. In *Proceedings of the 6th International Semantic Web Conference and 2nd Asian Semantic Web Conference, ISWC/ASWC 2007*, volume 4825 of *Lecture Notes in Computer Science*, pages 722–735. Springer, 2007. doi: 10.1007/978-3-540-76298-0_52. URL https://doi.org/10.1007/978-3-540-76298-0_52.
- [25] Robyn Speer, Joshua Chin, and Catherine Havasi. Conceptnet 5.5: An open multilingual graph of general knowledge. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence, AAAI 2017*, pages 4444–4451. AAAI Press, 2017. doi: 10.1609/aaai.v31i1.11164. URL <https://doi.org/10.1609/aaai.v31i1.11164>.
- [26] Halil Kilicoglu, Graciela Rosemblat, Marcelo Fiszman, and Thomas C. Rindflesch. SemMedDB: A PubMed-scale repository of biomedical semantic predications. *Bioinformatics*, 28(23):3158–3160, 2012. doi: 10.1093/bioinformatics/bts591. URL <https://doi.org/10.1093/bioinformatics/bts591>.
- [27] Prashant Garg and Thiemo Fetzer. Causal claims in economics. *CoRR*, abs/2501.06873, 2025. URL <https://arxiv.org/abs/2501.06873>.

- [28] Danilo Numeroso, Davide Bacciu, and Petar Veličković. Learning heuristics for A*. *CoRR*, abs/2204.08938, 2022. doi: 10.48550/arXiv.2204.08938. URL <https://doi.org/10.48550/arXiv.2204.08938>.
- [29] Sentence Transformers. sentence-transformers/all-mpnet-base-v2 model card. Hugging Face model card, <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>, 2022. Accessed: 2026-07-17.
- [30] BAAI. BAAI/bge-large-en-v1.5 model card. Hugging Face model card, <https://huggingface.co/BAAI/bge-large-en-v1.5>, 2023. Accessed: 2026-07-17.
- [31] IBM Granite Team. ibm-granite/granite-embedding-english-r2 model card. Hugging Face model card, <https://huggingface.co/ibm-granite/granite-embedding-english-r2>, 2025. Accessed: 2026-07-17.
- [32] Mixedbread AI. mixedbread-ai/mxbai-embed-large-v1 model card. Hugging Face model card, <https://huggingface.co/mixedbread-ai/mxbai-embed-large-v1>, 2024. Accessed: 2026-07-17.
- [33] Qwen Team. Qwen/Qwen3-Embedding-0.6B model card. Hugging Face model card, <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>, 2025. Accessed: 2026-07-17.
- [34] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, Joe Davison, Mario Šaško, Gunjan Chhablani, Bhavitvya Malik, Simon Brandeis, Teven Le Scao, Victor Sanh, Canwen Xu, Nicolas Patry, Angelina McMillan-Major, Philipp Schmid, Sylvain Gugger, Clément Delangue, Théo Matussière, Lysandre Debut, Stas Bekman, Pierric Cistac, Thibault Goehringer, Victor Mustar, François Lagunas, Alexander Rush, and Thomas Wolf. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-demo.21. URL <https://aclanthology.org/2021.emnlp-demo.21/>.
- [35] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International*

- Conference on Knowledge Discovery and Data Mining, KDD 2019*, pages 2623–2631. Association for Computing Machinery, 2019. doi: 10.1145/3292500.3330701. URL <https://doi.org/10.1145/3292500.3330701>.
- [36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems, NeurIPS 2019*, pages 8024–8035, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- [37] William Falcon and The PyTorch Lightning Team. PyTorch Lightning. Software, 2019. URL <https://doi.org/10.5281/zenodo.3828935>.
- [38] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2. URL <https://doi.org/10.1038/s41586-020-2649-2>.
- [39] Sebastián Ramírez. FastAPI. Software documentation, <https://fastapi.tiangolo.com/>, n.d. Accessed: 2026-07-08.
- [40] Vasco Asturiano. 3d-force-graph. Software repository, <https://github.com/vasturiano/3d-force-graph>, n.d. Accessed: 2026-07-08.
- [41] James S. Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24, pages 2546–2554. Curran Associates, Inc., 2011. URL https://proceedings.neurips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html.
- [42] Lutz Prechelt. Early stopping—but when? In Genevieve B. Orr and Klaus-Robert Müller, editors, *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer,

1998. doi: 10.1007/3-540-49430-8_3. URL https://doi.org/10.1007/3-540-49430-8_3.
- [43] Optuna Contributors. `optuna.pruners.MedianPruner`. Optuna documentation, <https://optuna.readthedocs.io/en/stable/reference/generated/optuna.pruners.MedianPruner.html>, 2026. Accessed: 2026-07-10.
- [44] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [45] PyTorch Contributors. `ReduceLROnPlateau`. PyTorch documentation, https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html, 2026. Accessed: 2026-07-10.
- [46] Myle Ott, Sergey Edunov, David Grangier, and Michael Auli. Scaling neural machine translation. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 1–9. Association for Computational Linguistics, 2018. doi: 10.18653/v1/W18-6301. URL <https://doi.org/10.18653/v1/W18-6301>.
- [47] Lightning AI. Gradient accumulation. Lightning documentation, https://lightning.ai/docs/pytorch/stable/common/gradient_accumulation.html, 2026. Accessed: 2026-07-17.
- [48] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. MS MARCO: A human generated machine reading comprehension dataset. *CoRR*, abs/1611.09268, 2016. URL <https://arxiv.org/abs/1611.09268>.
- [49] Iris Hendrickx, Su Nam Kim, Zornitsa Kozareva, Preslav Nakov, Diarmuid Ó Séaghdha, Sebastian Padó, Marco Pennacchiotti, Lorenza Romano, and Stan Szpakowicz. SemEval-2010 Task 8: Multi-way classification of semantic relations between pairs of nominals. In *Proceedings of the 5th International Workshop on Semantic Evaluation*, pages 33–38, Uppsala, Sweden, 2010. Association for Computational Linguistics. URL <https://aclanthology.org/S10-1006/>.
- [50] Rebecca Sharp, Mihai Surdeanu, Peter Jansen, Peter Clark, and Michael Hammond. Creating causal embeddings for question answering with

- minimal supervision. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 138–148, Austin, Texas, 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1014. URL <https://aclanthology.org/D16-1014/>.
- [51] Zhiyi Luo, Yuchen Sha, Kenny Q. Zhu, Seung won Hwang, and Zhongyuan Wang. Commonsense causal reasoning between short texts. In *Proceedings of the Fifteenth International Conference on Principles of Knowledge Representation and Reasoning*, pages 421–430. AAAI Press, 2016. URL <https://aaai.org/ocs/index.php/KR/KR16/paper/view/12818>.
- [52] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 641–649. Association for Computing Machinery, 2024. doi: 10.1145/3626772.3657878. URL <https://doi.org/10.1145/3626772.3657878>.
- [53] Parul Awasthy, Aashka Trivedi, Yulong Li, Meet Doshi, Riyaz Bhat, Vignesh P, Vishwajeet Kumar, Yushu Yang, Bhavani Iyer, Abraham Daniels, Rudra Murthy, Ken Barker, Martin Franz, Madison Lee, Todd Ward, Salim Roukos, David Cox, Luis Lastras, Jaydeep Sen, and Radu Florian. Granite embedding r2 models. *CoRR*, abs/2508.21085, 2025. URL <https://arxiv.org/abs/2508.21085>.
- [54] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems 33*, pages 16857–16867. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/c3a690be93aa602ee2dc0ccab5b7b67e-Abstract.html>.
- [55] Mixedbread AI. Open source strikes bread: New fluffy embedding model. Official technical blog, <https://www.mixedbread.com/blog/mxbai-embed-large-v1>, 2024. Published: 2024-03-08; accessed: 2026-07-17.
- [56] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 Embedding: Advancing text embedding and reranking through foundation models. *CoRR*, abs/2506.05176, 2025. URL <https://arxiv.org/abs/2506.05176>.

- [57] Jürgen Branke, Kalyanmoy Deb, Henning Dierolf, and Matthias Osswald. Finding knees in multi-objective optimization. In *Parallel Problem Solving from Nature – PPSN VIII*, volume 3242 of *Lecture Notes in Computer Science*, pages 722–731. Springer, 2004. doi: 10.1007/978-3-540-30217-9_73. URL https://doi.org/10.1007/978-3-540-30217-9_73.
- [58] Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. Finding a Kneedle in a haystack: Detecting knee points in system behavior. In *Proceedings of the 31st International Conference on Distributed Computing Systems Workshops*, pages 166–171. IEEE, 2011. doi: 10.1109/ICDCSW.2011.20. URL <https://doi.org/10.1109/ICDCSW.2011.20>.
- [59] Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395. Association for Computational Linguistics, 2004. URL <https://aclanthology.org/W04-3250/>.
- [60] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392. Association for Computational Linguistics, 2018. doi: 10.18653/v1/P18-1128. URL <https://aclanthology.org/P18-1128/>.

Use of AI Tools

OpenAI ChatGPT was used for language editing, formulation suggestions, consistency checks, and assistance with software development. All generated suggestions were reviewed, while responsibility for the scientific content, results, and cited sources remained entirely with the author.

Appendix A

Algorithmic Details

This appendix provides implementation-level details of the two variants of embedding-guided A^* search and the construction of search-aligned supervision. Algorithm 1 is the reachability-only variant used for binary evaluation. Because only a binary answer is required, it does not retain path information and returns as soon as the target is encountered among the successors of an expanded node. Algorithm 2 retains path information and is used when the discovered path itself is required, including supervision construction and interactive visualization. Algorithm 3 uses this path-returning variant without a visit budget to construct $\mathcal{D}_s^d$ for the training and validation splits. Further details and source code are provided in the accompanying repository.¹

¹<https://git.webis.de/code-teaching/theses/thesis-kiafard.git>

Algorithm 1 Bounded embedding-guided A^* reachability search

Require: Directed graph $G = (V, E)$, source node a , target node b , embedding function ϕ , distance function d , visit budget τ

Ensure: TRUE if a path from a to b is found, otherwise FALSE

```

1: if  $a = b$  then
2:   return TRUE
3: end if
4:  $open \leftarrow$  priority queue containing  $(0, 0, a)$ 
5:  $best\_g[a] \leftarrow 0$ 
6:  $closed \leftarrow \emptyset$ 
7: while  $open$  is not empty do
8:    $(f, g, u) \leftarrow$  entry with smallest priority from  $open$ 
9:   if  $u = b$  then
10:    return TRUE
11:   end if
12:   if  $u \in closed$  then
13:    continue
14:   end if
15:    $closed \leftarrow closed \cup \{u\}$ 
16:   if  $\tau \neq -1$  and  $|closed| > \tau$  then
17:    return FALSE
18:   end if
19:   if  $b \in N^+(u)$  then
20:    return TRUE
21:   end if
22:   for all  $v \in N^+(u)$  with  $v \notin closed$  do
23:      $g' \leftarrow g + d(\phi(u), \phi(v))$ 
24:     if  $v \notin best\_g$  or  $g' < best\_g[v]$  then
25:        $best\_g[v] \leftarrow g'$ 
26:        $h' \leftarrow d(\phi(v), \phi(b))$ 
27:       insert  $(g' + h', g', v)$  into  $open$ 
28:     end if
29:   end for
30: end while
31: return FALSE

```

Algorithm 2 Bounded embedding-guided A^* path search

Require: Directed graph $G = (V, E)$, source node a , target node b , embedding function ϕ , distance function d , visit budget τ

Ensure: A path from a to b if found, otherwise $\emptyset$

```

1:  $path\_links \leftarrow [(a, \text{NONE})]$ 
2:  $open \leftarrow$  priority queue containing  $(0, 0, a, 0)$ 
3:  $best\_g[a] \leftarrow 0$ 
4:  $closed \leftarrow \emptyset$ 
5: while  $open$  is not empty do
6:    $(f, g, u, i) \leftarrow$  entry with smallest priority from  $open$ 
7:   if  $u = b$  then
8:     return path reconstructed from  $path\_links$  at index  $i$ 
9:   end if
10:  if  $u \in closed$  then
11:    continue
12:  end if
13:   $closed \leftarrow closed \cup \{u\}$ 
14:  if  $\tau \neq -1$  and  $|closed| > \tau$  then
15:    return  $\emptyset$ 
16:  end if
17:  for all  $v \in N^+(u)$  with  $v \notin closed$  do
18:     $g' \leftarrow g + d(\phi(u), \phi(v))$ 
19:    if  $v \notin best\_g$  or  $g' < best\_g[v]$  then
20:       $best\_g[v] \leftarrow g'$ 
21:      append  $(v, i)$  to  $path\_links$ 
22:       $j \leftarrow |path\_links| - 1$ 
23:       $h' \leftarrow d(\phi(v), \phi(b))$ 
24:      insert  $(g' + h', g', v, j)$  into  $open$ 
25:    end if
26:  end for
27: end while
28: return  $\emptyset$ 

```

Algorithm 3 Construction of search-aligned supervision

Require: Causal pairs Q_s for split $s \in \{\text{train}, \text{val}\}$, causal graph $G = (V, E)$, base embedding model M , distance function $d \in \{\text{Cos}, \text{Euc}\}$

Ensure: Supervision dataset $\mathcal{D}_s^d$ of quadruples (c, e, p, n)

- 1: Precompute $\mathbf{z}_v \leftarrow M(v)$ for all $v \in V$
- 2: Define $\phi(v) \leftarrow \mathbf{z}_v$
- 3: $\mathcal{D}_s^d \leftarrow \emptyset$
- 4: $\text{cache} \leftarrow \emptyset$
- 5: **for all** $(A, B) \in Q_s$ **do**
- 6: Let $a, b \in V$ be the graph nodes corresponding to A, B
- 7: **if** $(a, b) \in \text{cache}$ **then**
- 8: $P \leftarrow \text{cache}[(a, b)]$
- 9: **else**
- 10: $P \leftarrow$ path from unbounded A^* on G from a to b using ϕ and d
- 11: $\text{cache}[(a, b)] \leftarrow P$
- 12: **end if**
- 13: **if** $|P| < 2$ **then**
- 14: **continue**
- 15: **end if**
- 16: **for** $i \leftarrow 0$ to $|P| - 2$ **do**
- 17: $c \leftarrow P_i$
- 18: $p \leftarrow P_{i+1}$
- 19: $e \leftarrow b$
- 20: **for all** $n \in N^+(c) \setminus \{p\}$ **do**
- 21: $\mathcal{D}_s^d \leftarrow \mathcal{D}_s^d \cup \{(c, e, p, n)\}$
- 22: **end for**
- 23: **end for**
- 24: **end for**
- 25: **return** $\mathcal{D}_s^d$
